REPORT · 2026年10月

交付文档

开发进展汇报 · GUI 使用手册 · 跨设备迁移指南

文保数字化研发组
2026年10月

目 录

一、开发进展总览 4

1.1 项目定位 4

1.2 版本演进 4

1.3 全流程阶段清单 4

二、本轮新增：跨尺度自适应 6

2.1 为什么要做 6

2.2 设计 6

2.3 哪些参数参与缩放 7

2.4 基准尺度的来龙去脉 7

2.5 开关与安全边界 7

2.6 第二层保护：点间距下限 8

2.7 顺带修复：配置模板污染 8

三、GUI 使用手册 9

3.1 启动方式 9

3.2 界面导览 9

3.2.1 欢迎页 9

3.2.2 三步向导 9

3.2.3 主界面三区 10

3.2.4 新增：文物尺度面板 10

3.3 命令行用法（无界面） 10

四、文件位置与目录结构 12

4.1 输入数据契约 12

4.2 输出产物 12

4.3 关键文件位置 12

五、跨设备迁移指南 14

5.1 先判断你属于哪种迁移 14
5.2 场景②：打包版迁移（推荐路径） 14
5.3 场景③：源码版迁移 14
5.4 COLMAP 依赖（无论哪种迁移都需要） 15
5.5 项目数据迁移 15
5.6 迁移后自检清单 16
5.7 迁移常见问题 16
六、质量验证与当前状态 **17**
6.1 本轮验证结果 17
6.2 尺度自适应实测量化 17
6.3 测试覆盖 18
6.4 当前已知限制 18
七、下一步方向 **19**
7.1 优先级排序 19
7.2 建议的下一步动作 19
附录 A：本批改动文件清单 **20**

一、开发进展总览

本文档面向三类读者：**项目负责人**（了解当前进度与风险）、**操作人员**（按 GUI 手册上手）、**部署人员**（跨设备迁移系统）。项目已进入 **v1.0.0 功能冻结 + 实机数据校验前夜**阶段：核心管线全部打通、测试体系完整、可执行程序已打包，唯一空白是**真实文物数据**。

1.1 项目定位

项目	内容
名称	H3D 造像类文物三维建模自动化系统
性质	硕士学位论文课题，**完全开源**
技术路线	**R2 全开源路线**：激光几何（自研）+ COLMAP 开源 SfM + 自研贴图管线
架构	分层：P0 核心底座 → L1 io 适配层 → L2 业务层 → M0 编排内核
设计约束	**严格适配器模式**：业务层不得直接 import Open3D，只能经 io 层接口
当前版本	v1.0.0（2026-08-30 冻结）+ 2026-09-15 加固 + 2026-10-09 跨尺度扩展

1.2 版本演进

阶段	内容	状态
v0.1 ~ v0.5	P0 底座 / L1 io / L2 preprocess / M2 配准 / M1-M6 合成验证 / COLMAP 路线迁移 / 照片纹理 / X1-X2 评估 / 论文表格	完成
v1.0.0（08-30）	首个正式版：M1-M6 + S1 + P1 + X1/X2 全流程，GUI 三步向导 + 一键生成 + 打包	完成
09-15 加固批	修 2 个 GUI BUG + 根治 4 项隐藏缺陷（多子重建取碎片 / 多线程不可复现 / 增量子命令名错误 / 副本过期缓存）；示例纹理重构（关键点 403 → 4248/图）；S1 稳定 16/16	完成
10-09 本批	新增**跨尺度自适应**：以包围盒对角线为基准线性缩放距离类参数，使同一套配置覆盖造像与中小型文物；修 to_yaml 浅拷贝污染缺陷	**本批交付**

1.3 全流程阶段清单

管线共 9 个阶段，按固定顺序调度（S1 照片位姿优先于 M1 几何，因 M1 需要消费 S1 的稠密产物）：

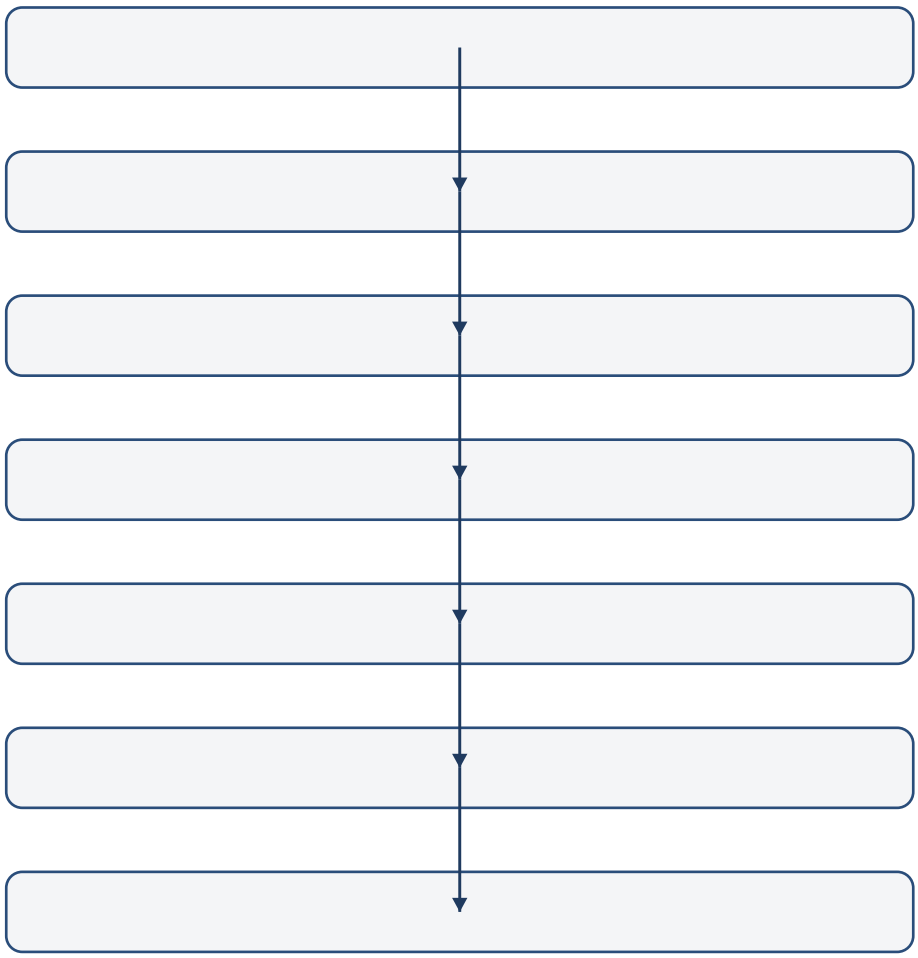


图 1-1 主管线阶段流（另有可选阶段 P1 照片纹理、X1 多时相形变、X2 色彩色差）

阶段	职责	核心产出目录
S1	照片位姿解算（COLMAP 封装 + 多子重建择优 + 特征参数自动标定）	99_runs/sfm_tmp/
M1	预处理：体素降采样 / 统计去噪 / 法线估计；激光 + dense 双源融合	10_clean/
M2	多站配准：法线补齐 → FPFH → RANSAC 粗配准 → ICP 精配准	20_registered/
M3	网格重建：Poisson / Ball-Pivoting / Alpha Shape 三选一	30_dense/
M4	纹理映射：UV 展开 + Z-buffer 可见性 + 多视图加权采样	50_tex/
M5	质检：几何覆盖率 / 中误差 + 纹理质量门（锐度 / 接缝）	60_qc/
M6	交付：多格式导出 + 元数据快照 + 交付清单 + 论文表格归档	80_export/ 90_meta/
P1	照片纹理 stage（capture_meta 位姿投影，可选）	50_tex/
X1/X2	多时相形变（M3C2）/ 色彩色差（ΔE，默认关闭）	60_qc/

二、本轮新增：跨尺度自适应

2.1 为什么要做

管线原先只面向**造像类**文物（0.5~3 m 量级），<code>preprocess</code> / <code>register</code> / <code>mesh</code> 中的**距离类参数是绝对米制值**：体素 5 mm、FPFH 半径 2 cm、ICP 距离 2 cm、RANSAC 距离 5 cm。这套值对造像合理，但遇到**中小型文物**（铜镜、印章、小型陶瓷/玉器，5~30 cm）就会退化：

参数	对 20 cm 器物的实际含义	后果
体素 5 mm	织物纹理级别被当作最小可分辨单元	吃掉纹饰细节，网格变粗糙
FPFH 半径 2 cm	等效「半个器身」的邻域	特征描述子趋同，特征退化
RANSAC 距离 5 cm	超过器物自身尺寸的对应距离	误匹配泛滥，粗配准失败
ICP 距离 2 cm	同上	精配准无法收敛

> **核心矛盾**：有意义的不是参数的**绝对值**，而是「**参数 / 物体尺寸**」之比。造像用 5 mm 体素 = 物体对角线的 0.13%；20 cm 器物要保持同等相对分辨率，体素应约为 **0.34 mm**——两者相差 15 倍。这就是尺度自适应的必要性。

2.2 设计

以**包围盒对角线**为唯一尺度基准，按比例线性缩放，使「参数 / 尺寸」之比恒定：

```
factor = clip(bbox_diag / reference_diag, min_factor, max_factor)
生效参数 = 配置参数 × factor
```

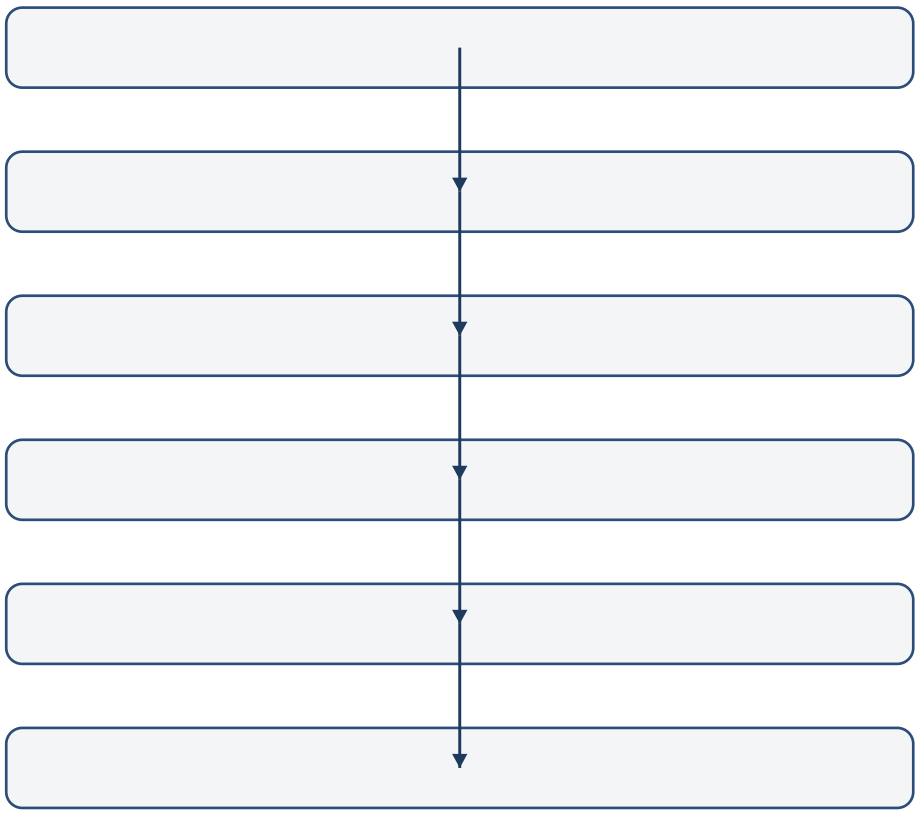


图 2-1 尺度解析流程（core.scale 纯数学层 → runner 桥接层）

2.3 哪些参数参与缩放

只缩放有长度量纲的字段。k 近邻数、比率、迭代次数、八叉树深度等**无量纲量保持不变**——因为体素随尺度缩放后，点密度相对物体恒定，这些量自洽。这是一条重要的设计边界，避免「为缩放而缩放」引入副作用。

配置段.字段	原值	量纲	是否缩放	理由
preprocess.voxel_down_sample	0.005 m	长度	**是**	点云分辨率基准
register.fpfh_radius	0.02 m	长度	**是**	特征邻域半径
register.ransac_distance	0.05 m	长度	**是**	粗配准对应距离
register.icp_distance	0.02 m	长度	**是**	精配准对应距离
mesh.alpha	0.05 m	长度	**是**	Alpha Shape 半径
mesh.ball_radii	[2,4,6] m	长度	**是**	BPA 球半径（逐元素）
change.core_radius	0.05 m	长度	**是**	M3C2 核心点半径
preprocess.normal_estimate_k	30	计数	否	k 近邻数，与尺度无关
register.max_iter	2000	计数	否	迭代次数
mesh.poisson_depth	8	深度	否	八叉树深度，与绝对尺度无关
quality.coverage_k_mps	3.0	倍数	否	MPS 倍数，已随点密度自适应

2.4 基准尺度的来龙去脉

<code>scale.reference_diag</code> 的语义是「**下面这套参数是在多大的物体上调出来的**」。默认值 **3.7 m** 的来历：示例造像的外接盒为 2.0 × 2.6 × 1.7 m，对角线 = 3.71 m——这正是现有默认参数的调参基准。取该值可保证**示例数据与历史项目的因子≈1.0（零回归）**。

对象	半轴尺寸	对角线	缩放因子	生效体素
大型造像	2.0 × 2.6 × 1.7 m	3.71 m	1.00	5.0 mm
中型文物	0.60 × 0.44 × 0.80 m	1.10 m	0.30	1.5 mm
小型文物	0.20 × 0.14 × 0.28 m	0.37 m	0.10	0.50 mm
微型文物	0.06 × 0.05 × 0.08 m	0.11 m	0.03	0.15 mm

若要迁移到**另一套标定体系**（例如你只在 0.5 m 的器物上调过参），把 <code>reference_diag</code> 改成对应尺度即可，其余参数无需逐个改。

2.5 开关与安全边界

配置项	默认	作用
scale.enabled	true	总开关。置 false 则完全沿用绝对值（历史行为）
scale.reference_diag	3.7	标定基准（米）——迁移标定体系时改它

配置项	默认	作用
scale.min_factor	0.005	因子下限（约 1.85 cm 的小件边界），防噪声点云把参数压到 0
scale.max_factor	5.0	因子上限（约 18.5 m），防单位错误把参数放大到失控
scale.warn_diag_gt	15.0	对角线超此值告警——提示「数据单位可能实为毫米」

三道安全网：① 开关可整体关闭；② 因子钳制区间防止极端值污染；③ 点云不可读 / 对角线无效时**自动退化为不缩放**并告警，绝不中断管线。另：<code>scale</code> 段自身**不参与缩放**（否则会递归失真）。

2.6 第二层保护：点间距下限

尺度缩放是**乘法**的，隐含假设「点间距随物体尺寸同比变化」。真实扫描通常满足（扫描分辨率固定、小物体拍摄距离近），但以下情形不满足：**网格均匀采样**生成的点云、**多精度设备混用**、或从照片稠密重建获得的**固定密度**点云。此时小尺度下会出现：

```
fpfh_radius × factor  <  实际点间距
→ 每点的邻域内只有它自己
→ FPFH 描述子退化为常数
→ 粗配准找不到对应关系，直接失败
```

修正为「取缩放结果与点间距下限的较大者」：

```
生效参数 = max(配置值 × factor，k × 点间距)
  k = 4.0（fpfh_radius / ransac_distance）
  k = 2.0（icp_distance）
```

点间距用 **1-NN 距离的中位数**估计（中位数比均值稳健，不受离群点与孔洞影响），采样 4000 点建 KDTree，开销可忽略。**对正常点云不产生任何改动**——因为常规情况下点间距远小于参数值，这正是零回归的保证。

2.7 顺带修复：配置模板污染

实现过程中暴露出一处**既有缺陷**：<code>gui/config_edit.py</code> 的 <code>to_yaml</code> 用 <code>dict(SCENE1_CONFIG)</code> **浅拷贝**模块级模板，随后对嵌套字典赋值（<code>cfg["scale"]["enabled"] = ...</code>）时**就地改写了全局模板**。后果：同一进程内第二次生成的配置会继承上一次的用户选择。已改为 <code>copy.deepcopy</code>，并补回归测试锁定。

三、GUI 使用手册

3.1 启动方式

场景	启动方式	说明
打包版（推荐给操作员）	双击 <code>H3D.exe</code>	免安装 Python 环境，自带全部依赖
源码版（开发调试）	<code>python -m h3d.gui</code> 或 <code>python -m h3d.cli gui</code>	需已激活项目虚拟环境
继承上次项目	<code>H3D.exe --batch <config.yaml></code>	无界面批处理模式，供脚本调用

打包版为 **onefile 单文件**（约 238 MB），首次启动解压到临时目录，冷启动约 5~15 秒属正常现象；第二次启动会快很多。

3.2 界面导览

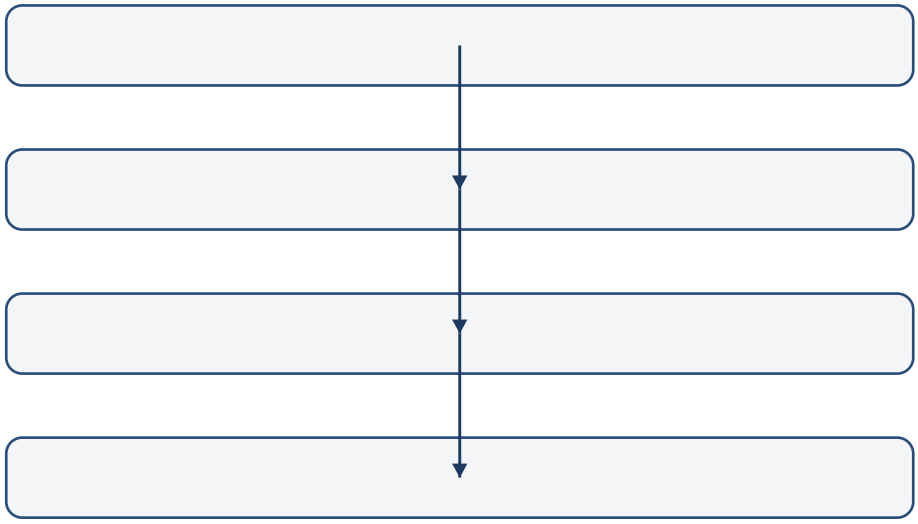


图 3-1 GUI 主流程

3.2.1 欢迎页

控件	作用
＋ 新建项目（向导）	进入三步向导，从零建立项目
▶ 运行示例项目（一键演示）	自动生成合成数据（激光点云 + 16 张环绕照片）并跑通全流程，**用于验收与演示**
最近项目列表	双击打开历史项目；右键删除记录
删除选中项目	移除历史记录（不删除磁盘文件）

> **首次使用建议先点「一键演示」**：无需准备任何数据，约 1~3 分钟即可看到完整交付物，用于确认软件在本机可正常运行。

3.2.2 三步向导

步骤	内容	要点
第一步：选择重建场景	场景① 激光几何 + 照片纹理（推荐） 场景② 纯照片三维重建（自动稠密重建）	有扫描仪点云选①；只有照片选②
第二步：导入数据 + 选择项目根目录	拖入激光点云（.ply/.pcd）、照片文件夹、控制点（选填）	**照片必须拖入文件夹**，不支持单张；建议 ≥30 张环绕拍摄
第三步：参数确认	项目名称 / 纹理分辨率 / 输出格式	高级参数（降采样、泊松深度、**文物尺度**等）可稍后在主界面折叠面板调整

第二步的数据校验会实时提示：照片过少（<3 张）会警告、拖入的不是文件夹会明确报错并说明支持格式，避免带着错误配置进入长耗时流程。

3.2.3 主界面三区

区域	内容	说明
配置区	基础参数 + **高级参数折叠面板**	高级面板含「点云与网格」「纹理与融合」「重建与质量门」「**文物尺度**」四组
预览区	标签页：3D 模型预览 / 纹理查看	另有「打开 open3d 交互窗口（真 3D）」与「X1 形变热力图」按钮
状态区	**总进度条 + 9 个阶段状态灯 + 实时日志**	状态灯：S1 M1 M2 M3 M4 M5 M6 X1 X2，颜色表示 等待/运行/完成/失败

按钮	作用
一键生成交付物	按当前配置启动全流程（后台线程，界面不卡死）
暂停 / 取消	中断当前运行（已完成的阶段产物保留，可续跑）
导出完整交付包	把模型 + 纹理 + 质检报告 + 元数据打包为一个交付目录

3.2.4 新增：文物尺度面板

本轮在高级面板新增「**文物尺度**」分组，这是覆盖中小型文物的关键控件：

控件	默认	说明
按几何尺度自动调整参数	勾选	勾选后体素 / FPFH 半径 / RANSAC 与 ICP 距离会按点云包围盒对角线自动缩放。**取消勾选则完全沿用绝对值**
参数基准尺度(m)	3.7	「下面那套参数是在多大文物上调出来的」。默认 3.7 对应示例造像；**仅当迁移到另一套标定体系时才需修改**

对**常规使用**而言，保持默认（勾选自动、基准 3.7）即可——系统会自动识别你导入的是 3 m 的造像还是 20 cm 的铜镜。

3.3 命令行用法（无界面）

```
h3d init     -c config.yaml     # 初始化项目目录结构
h3d validate -c config.yaml     # 校验配置合法性与数据完整性（不处理）
h3d batch    -c config.yaml     # 批量处理全部文物（无人值守）
h3d report   -c config.yaml     # 仅重新生成报表
h3d gui                         # 启动图形界面
```

打包版等价写法：<code>H3D.exe --batch <config.yaml></code>。批处理模式下日志同时写入项目根目录的 <code>batch_gui.log</code> 与标准输出。

四、文件位置与目录结构

4.1 输入数据契约

项目根目录下，**每个子目录 = 一件文物**（目录名即文物编号）：

目录	内容	必需性	格式
<code>00_raw/<编号>/</code>	几何源（激光点云 或 dense 点云）	**必需**	ply / pcd / obj
<code>00_photo/<编号>/</code>	照片（S1 位姿 + M4 纹理共用源）	**必需**（≥2 张）	jpg / png / tif
<code>00_ctrl/<编号>/</code>	控制点（两阶段对齐）	可选	JSON
<code>00_ref/<编号>/</code>	参考数据（X1 形变对比基准）	可选	ply
<code>90_meta/</code>	capture_meta.json 相机位姿	可选	JSON

若自带相机位姿（相机阵列 / 已标定场景），可提供 <code>capture_meta.json</code> 让 M4 跳过 S1 直接投影。其结构为 <code>{n_cameras, n_images, cameras, images}</code>，**其中 t 必须是相机中心**（前方 -Z）；若从 COLMAP 导出，需经 <code>C = -R_co□ · tvec</code> 转换，否则纹理会整张空白。

4.2 输出产物

目录	内容
<code>10_clean/</code>	M1 清洗后的分站点云
<code>10_dense/</code>	S1 稠密点云（fused_dense.ply 供 M1 双源融合）
<code>20_registered/</code>	M2 统一坐标系下的多站点云
<code>30_dense/</code>	M3 网格模型
<code>50_tex/</code>	M4 带纹理模型 + 纹理图集 PNG
<code>60_qc/</code>	M5 质检报告与图件
<code>70_report/</code>	报表三件套（CSV / PNG / HTML）
<code>80_export/</code>	M6 交付模型（obj / ply / glb）
<code>90_meta/</code>	元数据快照、交付清单、相机参数
<code>99_runs/</code>	S1 COLMAP 工作区（database.db / sparse / photos_small）
<code>output/run_manifest.json</code>	运行清单（阶段状态 + 指标 + 耗时，支撑续跑）
<code>report/</code>	HTML 报告（浏览器打开）

4.3 关键文件位置

用途	路径
打包版可执行文件	<code>D:/h3d_relic_test/pyi_dist_v3/H3D.exe</code>
安装包脚本	<code>packaging/H3D_installer.iss</code>（Inno Setup）
源码根目录	<code>E:/h3d_lablab/</code>

用途	路径
Python 环境	<code>E:/h3d_lablab/.venv_lab/Scripts/python.exe</code>
打包配置	<code>h3d_gui.spec</code>
配置模板	<code>configs/example.yaml</code>
COLMAP 可执行文件	<code>E:/colmap/colmap-x64-windows-cuda/bin/colmap.exe</code>
工作记录	<code>.workbuddy/memory/</code>

五、跨设备迁移指南

5.1 先判断你属于哪种迁移

场景	是否需要 Python/COLMAP	难度	推荐做法
① 同机换目录	否	低	直接拷贝整个项目根目录
② 换机、仅用软件跑数据	**否**	低	**拷贝 H3D.exe（免安装）**
③ 换机、需改代码	是	中	重建虚拟环境 + 安装 COLMAP
④ 换机、无 NVIDIA 显卡	是	中高	改用 CPU 版 COLMAP，关闭 CUDA 相关项

> **如果只是要「在另一台电脑上跑数据」，场景②最省事**：<code>H3D.exe</code> 是 onefile 打包，已经把 Python 解释器、PySide6、open3d、numpy 等全部依赖打进单文件，目标机**无需安装任何环境**，复制过去双击即可。

5.2 场景②：打包版迁移（推荐路径）

步骤：

```
1) 拷贝 D:/h3d_relic_test/pyi_dist_v3/H3D.exe 到目标机任意目录
2) 目标机需安装 COLMAP（见 5.4）—— 因为 COLMAP 是外部程序，
   不包含在 exe 内
3) 在项目 config.yaml 中把 sfm_solve.colmap_path 改为目标机的
   COLMAP 实际路径
4) 双击 H3D.exe 启动，先跑「一键演示」验证环境
```

必须注意：<code>config.yaml</code> 里的 <code>sfm_solve.colmap_path</code> 是**机器相关绝对路径。**GUI 生成的配置默认写死为开发机的 <code>E:/colmap/colmap-x64-windows-cuda/bin/colmap.exe</code>，迁移后**必须改成目标机的实际路径**，否则 S1 阶段会失败。

5.3 场景③：源码版迁移

目标机需具备 **Python 3.10+**（推荐 3.12）。步骤：

```
# 1) 拷贝源码（排除虚拟环境与构建产物）
robocopy E:\h3d_lablab D:\h3d_lablab /E \
    /XD .venv_lab build_work build_work_v2 dist build_out __pycache__ \
    /XD output\_demo_full output\_demo_full2 output\_demo_scale

# 2) 重建虚拟环境
cd D:\h3d_lablab
python -m venv .venv_lab
.venv_lab\Scripts\python.exe -m pip install -U pip
.venv_lab\Scripts\pip.exe install -r requirements.txt

# 3) 以可编辑模式安装本项目（提供 h3d 命令）
.venv_lab\Scripts\pip.exe install -e .

# 4) 自检：跑测试套件
.venv_lab\Scripts\python.exe -m pytest tests/ -q
```

依赖	版本要求	备注
Python	3.10+（推荐 3.12）	项目在 3.12.10 上验证
numpy / scipy	≥1.26 / ≥1.12	数值基础
open3d	≥0.19	**仅经 L1 io 层调用**（适配器模式）
PySide6	≥6.11	GUI
cupy-cuda12x	≥13.0	**仅在需要 GPU 加速时安装**，与 COLMAP CUDA 12 对齐
matplotlib / Pillow / opencv	见 requirements	绘图与图像
pyinstaller	≥6.0	仅重新打包时需要

5.4 COLMAP 依赖（无论哪种迁移都需要）

S1 位姿解算依赖外部程序 **COLMAP**（BSD-3 许可，属开源路线的一部分）。项目**不打包 COLMAP**，需在目标机单独安装：

目标机情况	推荐版本	下载要点
有 NVIDIA GPU + CUDA	**colmap-x64-windows-cuda**	CUDA 版，S1 稠密重建快 5~10 倍
无 GPU / 集显	**colmap-x64-windows-nocuda**	纯 CPU 版，功能完整但慢；须在配置中把 GPU 相关项置 0

安装后在 <code>config.yaml</code> 中指向实际路径（注意 Windows 路径用正斜杠或双反斜杠）：

```
sfm_solve:
  enabled: true
  colmap_path: 'D:/colmap/bin/colmap.exe'   # ← 改成目标机实际路径
  matcher: sequential
  auto_calibrate: true
  deterministic: true    # 论文级可复现（单线程，大图集耗时上升）
```

5.5 项目数据迁移

项目根目录包含大量中间产物，**可以按需精简。**如果只是迁移「原始数据 + 配置」，只保留以下内容即可，目标机上重新跑一遍会重建全部产物：

保留	可删除	说明
<code>00_raw/</code> <code>00_photo/</code> <code>00_ctrl/</code> <code>00_ref/</code>	10_clean 20_registered 30_dense 50_tex 60_qc 70_report 80_export	中间产物可完全重建
<code>config.yaml</code>	—	迁移后需改 colmap_path
—	<code>99_runs/</code>	COLMAP 工作区，体积最大（含 photos_small 与 DB）
—	<code>output/run_manifest.json</code>	**删除它 = 强制全量重跑**（不删则跳过已完成阶段）

续跑机制提醒：<code>output/run_manifest.json</code> 记录了各阶段完成状态。迁移后若想**从零重跑**，删除该文件即可；若想**保留已完成阶段**，需同时保留对应产物目录，否则产物缺失会触发该阶段及下游级联重跑。

5.6 迁移后自检清单

#	检查项	预期结果
1	双击 <code>H3D.exe</code> 能打开欢迎页	界面正常显示（**此条可验证 Python 依赖完整**）
2	点「运行示例项目（一键演示）」	约 1~3 分钟跑完，S1 显示 **16/16 注册**
3	查看 9 个阶段灯	S1~M6 全绿，X1/X2 灰（默认关闭）
4	打开 <code>report/report.html</code>	报表可正常渲染，含阶段耗时与质检指标
5	用自备真实数据跑一次	重点看 **S1 注册率**与 **M5 纹理覆盖率**
6	检查日志中的「尺度自适应」行	显示包围盒对角线与缩放因子

5.7 迁移常见问题

现象	原因	处理
双击 exe 无反应 / 闪退	onefile 解压被杀软拦截，或临时目录无权限	加入杀软白名单；从命令行运行 <code>H3D.exe --batch x.yaml</code> 看报错
S1 阶段报错找不到 colmap	config.yaml 中的 colmap_path 仍是开发机路径	改成目标机 COLMAP 实际路径（**最常见问题**）
S1 报 CUDA / cublas 相关错误	装了 CUDA 版 COLMAP 但目标机无 NVIDIA 驱动	换 CPU 版 COLMAP，并在配置中关闭 GPU 相关项
S1 注册数偏低（如 3/16）	照片重叠不足 / 纹理过弱；或读到了旧的工作区	提高重叠度至 70%+；删除 <code>99_runs/</code> 强制重建
换了照片但结果不变	历史版本曾因预处理副本过期缓存导致	**2026-09-15 已修复**；更新到最新 exe 即可
界面显示但点「一键生成」无进度	历史版本曾因 stdout 未回传（已修复）	更新到最新 exe；查看 <code>batch_gui.log</code> 是否有内容
中文路径下失败	个别外部程序不兼容非 ASCII 路径	把项目根目录与 COLMAP 路径放在纯英文路径下
大图集 S1 很慢	deterministic=true 强制单线程（为保证可复现）	若不需要比特级复现，可在配置中置 <code>deterministic: false</code>

六、质量验证与当前状态

6.1 本轮验证结果

验证项	方法	结果
尺度解析正确性	4 个尺度档位（造像/中型/小型/微型）逐项核对因子与生效参数	**通过**——因子与对角线严格成正比
示例零回归	一键演示全流程，比对生效参数与历史值	因子 **1.0018**，参数仅微调 0.18%，**无实质影响**
小件自适应生效	同配置跑不同尺度数据，对比 M1 保留点数与配准 fitness	见 6.2
全量回归测试	<code>pytest tests/ -q</code>	**443 passed**
全流程贯通	一键演示 batch 模式	**done=1，EXIT=0**，报表三件套齐全
S1 注册率	示例 16 张环绕照片	**16/16（100%）**
配置模板隔离	连续两次生成配置，检查全局模板是否被污染	**通过**——深拷贝修复已生效

6.2 尺度自适应实测量化

最直接可量化的效果在 **M1 分辨率保真度**上。同一份 60000 点点云，分别以「关闭自适应」与「开启自适应」跑预处理，统计体素降采样后的剩余点数：

对象	对角线	因子	生效体素	剩余点数(关)	剩余点数(自)	保真提升
大型造像	3.941 m	1.0650	5.33 mm	55335	55020	1.0×
中型文物	1.175 m	0.3175	1.59 mm	35782	54971	1.5×
小型文物	0.400 m	0.1080	0.54 mm	7301	54783	**7.5×**
微型文物	0.120 m	0.0325	0.16 mm	774	55162	**71.3×**

> **关键证据**：关闭自适应时，微型文物的 60000 点被体素降采样压到只剩 **774 点**——器物基本只剩下一个粗略轮廓，纹饰细节全部丢失；开启自适应后保留 **55162 点**（**71.3 倍**）。大型造像则几乎无变化（1.0×），**证明零回归。**

从「参数 / 尺寸」比例看更直观——自适应的本质是**让比例恒定**：

对象	FPFH 半径(关)	占对角线	FPFH 半径(自)	占对角线	说明
大型造像	20 mm	0.51%	21.3 mm	0.54%	两者均合理
中型文物	20 mm	1.70%	6.34 mm	0.54%	关闭时偏大
小型文物	20 mm	**5.00%**	2.16 mm	0.54%	关闭时严重超标
微型文物	20 mm	**16.7%**	0.65 mm	0.54%	关闭时等效「整个器身」

FPFH 邻域半径的合理量级是物体尺寸的 **1~2%**。关闭自适应时该比例随物体变小而**急剧膨胀**（微型达 16.7%，等价于每点的邻域覆盖整个器物，特征描述子趋同、丧失判别力）；开启后**恒定在 0.54%**。这正是「同一套配置无法覆盖不同尺度」的量化根源。

> **诚实边界**：M2 配准层面的自适应效果**本批未能用合成数据验证**。实验显示合成光滑表面上「关闭自适应反而配准更准」——因为光滑表面的**局部** FPFH 判别力弱，半径放大后邻域覆盖更多几何上下文，反而形成类似「全局形状描述子」的强特征。真实扫描点云（有噪声、有细节、密度随尺度变化）的特性与此**相反**。因此配准效果**必须由真实数据确认**，本条已列入 6.4 已知限制。

6.3 测试覆盖

测试文件	新增内容	结果
<code>tests/test_core/test_scale.py</code>	<code>TestBboxDiagonal</code>(7)、<code>TestResolveFactor</code>(9)、<code>TestScaledConfig</code>(9)、<code>TestSpacingFloor</code>(8)、<code>TestTaskScaleIntegration</code>(5)、<code>TestNoRegressionOnDemoScale</code>(1)、<code>TestGuiScaleConfig</code>(3)	**41 passed**
全量	覆盖 core / io / preprocess / register / mesh / quality / runner / report / gui / change 等全部模块	**443 passed**

测试断言锁「行为本质」而非「不报错」：例如 <code>test_proportional_scaling</code> 断言因子与对角线严格成正比、<code>test_source_config_not_mutated</code> 断言缩放不污染原配置、<code>test_dimensionless_fields_untouched</code> 断言 k 值与深度不被误缩放、<code>test_floor_noop_for_normal_dense_cloud</code> 断言点间距下限对正常点云**零改动**（零回归保证）、<code>test_template_not_polluted_by_to_yaml</code> 锁定模板隔离。

6.4 当前已知限制

限制	影响	后续
实机真实数据未验证	**最高风险**：所有验证基于合成示例	下一步核心工作，见第七章
配准层面的自适应效果未在合成数据上验证	合成光滑表面与真实扫描特性**相反**（详见 6.2 末），M2 的实际收益需真实点云确认	实机校验时对比关闭/开启两种模式的 fitness 与角误差
S1 单相机假设	写死 <code>--ImageReader.single_camera 1</code>，多机位/变焦拍摄会标定失败	按 EXIF 分组或置 0
M5 纹理覆盖率 29.1%	对示例规模偏低（目标是 60%+）	复核 UV 图集利用率与 Z-buffer 剔除阈值
示例仍产出 1 个 2 张的丢弃子重建	已被择优逻辑正确跳过，不影响结果	可增相邻视角重叠彻底消除
确定性模式耗时	<code>deterministic=true</code> 单线程，大图集慢	仅在需要比特级复现时开启

七、下一步方向

7.1 优先级排序

优先级	事项	说明
高	真实文物照片实机校验	唯一能推翻现有结论的动作。重点观察：子重建数量、S1 注册率、M5 纹理覆盖率
高	采集规范落地	小件 40~80 张、重叠 ≥70%、锁定焦距与曝光、画面内放标尺（尺度可追溯）
中	M4 纹理覆盖率提升	29% → 60%+；查 UV 图集利用率与 Z-buffer 可见性阈值
中	多机位支持	放开单相机假设，按 EXIF 分组标定，适配换镜头拍摄细节的场景
中	论文材料定型	所有对比实验固定 <code>deterministic=true</code> 并注明参数；把 S1 可复现性排查过程写为方法论章节
低	材质反射对策	青铜/陶瓷/玉器的镜面区会使 SIFT 失效——属采集端问题（偏振片 / 漫反射罩）
低	打包分发收尾	用 <code>packaging/H3D_installer.iss</code> 制作安装包；补目标机降级路径说明

7.2 建议的下一步动作

拿一组真实造像照片（8~20 张，重叠 70%+）与对应的扫描点云，用最新 <code>H3D.exe</code> 跑一次批处理，把 **S1 日志**与 **M5 质量指标**拿出来分析。这一步的产出决定后续三条线的优先级排序——如果实机 S1 注册率掉到 60% 以下，第一项会吞掉其余项的排期。

同时建议在**同一台机器**上先用中小型文物（如 20 cm 铜镜）做一次对照：由于本轮已加入尺度自适应，配置可完全复用，只需把数据放入 <code>00_raw/<编号>/</code> 与 <code>00_photo/<编号>/</code> 即可。

附录 A：本批改动文件清单

文件	类型	改动
<code>src/h3d/core/scale.py</code>	**新增**	尺度解析：bbox_diagonal / resolve_factor / scaled_config / estimate_spacing / apply_spacing_floor / describe
<code>src/h3d/core/config.py</code>	修改	新增 <code>ScaleConfig</code> 段并注册进 <code>PipelineConfig</code>
<code>src/h3d/core/__init__.py</code>	修改	导出 ScaleConfig 与 scale 函数
<code>src/h3d/runner/task.py</code>	修改	Task 增加 scale_factor / scale_diag / scale_info / scale_changes 字段
<code>src/h3d/runner/batch_processor.py</code>	修改	新增 probe_geometry_diagonal / resolve_task_config / _record_scale / _inject_scale_metrics；单任务与并行 worker 两条调度路径接入
<code>src/h3d/gui/config_edit.py</code>	修改	模板加 scale 段；ProjectConfig 加 autoscale / ref_diag；**to_yaml 改深拷贝修复模板污染**
<code>src/h3d/gui/advanced.py</code>	修改	新增「文物尺度」分组
<code>configs/example.yaml</code>	修改	新增 scale 段并加注释说明
<code>tests/test_core/test_scale.py</code>	**新增**	41 项回归测试
<code>output/_t_scale_verify.py</code>	**新增**	尺度效果对照验证脚本